

UNIT-IV

RDBMS

- It stands for Relational Database Management System.
- RDBMS is software used to store, manage & retrieve data in the form of tables (rows & columns) based on the relational model.

Key Features • Tables (Relations): - Data is organized into tables.

- Rows (Tuples): - Individual records.
- Columns (Attribute): - Data fields.
- Primary Key: - Uniquely identifies each row.
- Foreign Key: - Establishes relationships between tables.
- SQL: - Used to create, query, update & delete data.
- ACID properties: Ensures reliable transactions:
 - Atomicity
 - Consistency
 - Isolation
 - Durability

Summary :



Advantages • Reduces data redundancy through normalization.

- Maintains data integrity.
- Supports complex queries & relationships.
- Provides security & transaction management.

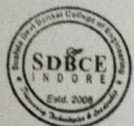
Popular RDBMS Examples • MySQL

- PostgreSQL
- Oracle Database
- Microsoft SQL ~~Server~~ Server
- SQLite

Difference Between DBMS & RDBMS

S.No.	DBMS	RDBMS
1.)	May stores data as files	Stores data in related tables
2.)	Relationships not mandatory	Relationships enforced through keys
3.)	Less suitable for large systems	Suitable for large, multi-user systems.
4.)	Examples: dBase, FoxPro	MySQL, PostgreSQL, Oracle.

Summary :



Functional Dependency

A functional dependency is a relationship between two attributes in a relation where the value of one attribute uniquely determines the value of another attribute.

Notation :

$$A \rightarrow B$$

It means if we know the value of A, we can uniquely determine the value of B.

Ex: - (1) Roll No. $\rightarrow$ Name

Ex: - (2) Consider a Student table:

Student-ID	Student-Name	Department
101	Ankur	CSE
102	Mayank	IT
103	Palak	AIML

Here: • Student-ID $\rightarrow$ Student-Name
• Student-ID $\rightarrow$ Department

Summary :

Because each Student-ID ~~to~~ uniquely identifies one student and their department.

Trivial Functional Dependency

A functional dependency is called trivial if the attribute on the right-hand side (RHS) is already present in the left-hand side (LHS).

Ex:- $(A, B) \rightarrow A$

$(A, B) \rightarrow B$

$RHS \subseteq LHS$

Here, A & B are already part of the left-side, so there are trivial FD.

Non-Trivial Functional Dependency

A functional dependency is called non-trivial if the attribute on the right-hand side is not present in the left hand side.

Ex:- Roll No. $\rightarrow$ Name

$EMPID \rightarrow EmpName$

Summary :



Here, Name & Emp-name are not present on the left side, so these are Non-Trivial FD.

RHS $\neq$ LHS

Completely Non-Trivial Functional Dependency

A functional dependency is completely non-trivial if there is no common attribute between X & Y.

Form:- $X \cap Y = \emptyset$

Ex:-

- Roll-No $\rightarrow$ Name
 - Employee-ID $\rightarrow$ Salary
- No attribute appears on both sides.

Full Functional Dependency

An attribute is fully functionally dependent on a set of attributes if it depends on the entire set & not on any subset of it.

Ex:- Student-Course (Roll-No, Course-ID, Grade)

- (Roll-No, Course-ID) $\rightarrow$ Grade

Grade depends on both Roll-No & Course-ID together.

Summary :



Partial Dependency

A dependency is partial if an attribute depends on only a part of a composite key.

Ex:- Student-Course (Roll-No, Course-ID, Student-Name)

- (Roll-No, Course-ID) $\rightarrow$ Student-Name
- Roll-No $\rightarrow$ Student-Name

Student-Name depends only on Roll-No, not on the whole key.

Transitive Dependency

A dependency is transitive when one attribute depends on another non-key attribute.

Ex:- Employee (Employee-ID, Dept-ID, Dept-Name)

- Employee-ID $\rightarrow$ Dept-ID
- Dept-ID $\rightarrow$ Dept-Name

— This is a transitive dependency.

Summary :



Closure of FD Set (F^+)

The Closure of a FD set (F^+) is the set of all functional dependencies that can be derived from a given set of functional dependencies F , using Armstrong's Rules.

Ex:- Given

$$A \rightarrow B$$

$$B \rightarrow C$$

Then we can derive :-

$$A \rightarrow C$$

So,

$$F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$$

Closure of Attributes (A^+)

The attribute closure of an attribute set A is the set of all attributes that can be functionally determined from A .

It is used to find:

- 1.) Candidate Key
- 2.) Super key
- 3.) Functional Dependencies

Summary :



Ex:- Given FD:

$$A \rightarrow B$$

$$B \rightarrow C$$

$$C \rightarrow D$$

Find A^+

Start with :

$$A^+ = \{A\}$$

Using $A \rightarrow B$

$$A^+ = \{A, B\}$$

Using $B \rightarrow C$

$$A^+ = \{A, B, C\}$$

Using $C \rightarrow D$

$$A^+ = \{A, B, C, D\}$$

— Therefore :

$$A^+ = \{A, B, C, D\}$$

Summary :



Difference between FD Closure & Attribute Closure

S.No.	FD Closure	Attribute Closure
1.)	All possible functional dependencies	All possible Attributes
2.)	Written as F^+	Written as A^+
3.)	Used to derive new FD	Used to find Candidate Key.
4.)	<u>Ex:-</u> $A \rightarrow C$ derived from $A \rightarrow B, B \rightarrow C$	<u>Ex:-</u> $A^+ = \{A, B, C, D\}$

Irreducible set of FD

- It is also known as Irreducible set / Minimum FD Set / Minimal Cover.
- An Irreducible set of functional dependencies is a set of FD in which:

1.) Every FD has only one attribute on the RHS.

2.) No FD is redundant.

3.) No attribute on the LHS is unnecessary.

In other words, it is the smallest equivalent set of FDs.

Summary :



Ex:- Given FD set:

$$A \rightarrow BC$$

$$B \rightarrow C$$

$$A \rightarrow B$$

Step 1:- Split RHS

$$A \rightarrow B$$

$$A \rightarrow C$$

$$B \rightarrow C$$

Step 2:- Remove Redundant FD

Since $A \rightarrow B$

$$B \rightarrow C$$

Already imply:

$$A \rightarrow C$$

Therefore:

$$A \rightarrow C$$

is redundant & can be removed.

Minimal Cover

$$A \rightarrow B$$

$$B \rightarrow C$$

— This is the Irreducible Set of FD.

Summary :



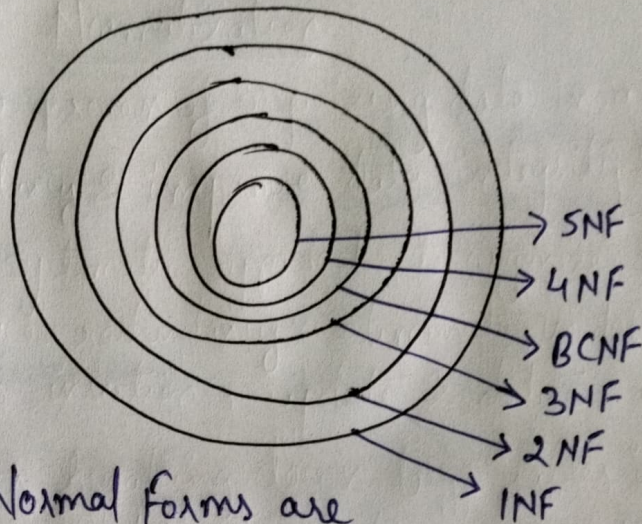
Normalization

- Normalization is the process of organizing data in a database to minimize redundancy & improve data integrity & consistency.
- Normalization is a systematic approach to organize data within a database to reduce redundancy & eliminate undesirable characteristics such as insertion, update & delete anomalies.
- The process involves breaking down large tables in smaller, well structured ones & defining relationships between them.
- This not only reduces the chances of storing duplicate data but also improves the overall efficiency of the database.

Why is Normalization Important?

- 1.) Reduces Data Redundancy
- 2.) Improves Data Integrity
- 3.) Simplifies Database Design
- 4.) Optimizes Performance

Summary :



Normal Forms :- Normal forms are the different stages or levels of normalization used to organize data in a database a/c to specific rules.

1NF

- A relation is in 1NF if every attribute contains only atomic (single) values & there are no repeating groups.
- ~~Table~~ One cell contains one value.

• Table

R#	Name	Subjects
101	Palak	DBMS, BCE

Correct Table

R#	Name	Subjects
101	Palak	DBMS
101	Palak	BCE

Summary : a/c $\Rightarrow$ according



2NF

- A relation is in 2NF if it is in 1NF & every non-key attribute is fully functionally dependent on the whole primary key.
- Remove partial dependency.

3NF

- A relation is in 3NF if it is in 2NF & there is no transitive dependency among non-key attributes.
- No Transitive Dependency.

BCNF

- It stands for Boyce-Codd Normal Form.
- BCNF is between 3NF & 4NF.
- A relation is in BCNF if, for every non-trivial functional dependency ($X \rightarrow Y$), X is a super key.
- A Table is in BCNF if the left side (determinant) of every functional dependency is a super key.

4NF

- A relation is in 4NF if it is in BCNF & contains no non-trivial multivalued dependency.

Summary :



SNF.

A relation is in SNF if it is in 4NF & contains no join dependency except those implied by candidate keys.

UNF $\Rightarrow$ Unnormalized Form



1NF $\Rightarrow$ Repeating values remove



2NF $\Rightarrow$ Partial dependency remove



3NF $\Rightarrow$ Transitive dependency remove



BCNF $\Rightarrow$ Every Determinant must be a super key.



4NF $\Rightarrow$ Multivalued Dependency remove



SNF $\Rightarrow$ Join Dependency remove

Multivalued Dependency

A multi-valued dependency exists when one attribute determines independent values of another attribute.

Join Dependency

A join dependency exists when a relation can be reconstructed by joining three or more decomposed relations.

Summary :



Sushila Devi Bansal College of Engineering

Umariya, A.B. Road, Indore

Page No. 94.

Below is one comprehensive example that can be converted from 1NF to SNF.

Original Table (UNF)

Student-ID	Student-Name	Course-ID	Teacher	Hobby
S101	Rahul	C101	Amit	Cricket, music

1NF

Student-ID	Student-Name	Course-ID	Teacher	Hobby
S101	Rahul	C101	Amit	Cricket
S101	Rahul	C101	Amit	Music

2NF

Student

Student-ID	Student-Name
S101	Rahul

Enrollment

Student-ID	Course-ID
S101	C101

Course

Course-ID	Teacher
C101	Amit

Summary :



Sushila Devi Bansal College of Engineering

Umariya, A.B. Road, Indore

Page No. 95.

Student - Hobby

Student-ID	Hobby
S101	Cricket
S101	Music

3NF

Teacher	Course-ID
Amit	C101

Course	Course-ID
	C101

BCNF

Teacher	Course-ID
Amit	C101

4NF

Student-Hobby	Student-ID	Hobby
	S101	Cricket
	S101	Music

5NF

Student - Course

Teacher - Course

Student - Teacher

Student-ID	Course-ID
S101	C101

Teacher	Course-ID
Amit	C101

Student-ID	Teacher
S101	Amit

Summary :



Decomposition using FD preservation

Dependency preservation means that after decomposing a relation into smaller relations, all the original functional dependencies (FDs) can still be enforced without joining the relations.

Ex:- Relation

$R(A, B, C)$

FDs $A \rightarrow B$

$B \rightarrow C$

Divided into 2 table

$R_1(A, B)$

$R_2(B, C)$

A	B
1	X
2	Y

B	C
X	P
Y	Q

Then, $A \rightarrow B$
 $B \rightarrow C$

Lossless Join

A decomposition is called a lossless join decomposition if the original relation can be reconstructed exactly by joining the decomposed tables. No information is lost & no extra tuples are generated.

Summary :



Ex:- Original Relation :

Roll No.	Name	Dept
101	Aman	CSE
102	Neha	IT

Decompose into :

Student

Roll No.	Name
101	Aman
102	Neha

Dept.

Roll No.	Dept
101	CSE
102	IT

Join on Roll-No

Roll-No.	Name	Dept.
101	Aman	CSE
102	Neha	IT

Lossy Join

A decomposition is called a Lossy Join decomposition if joining the decomposed tables does not reproduce the original relation exactly.

Summary :



Some information is lost or extra tuples (Spurious) are generated.

- In lossy join decomposition extra tuple is also known as Spurious.

Ex:- Original Relation:

Name	Dept.	Subject
Aman	CSE	DBMS
Neha	IT	Java

Decompose into:-

R_1 (Name, Dept)

Name	Dept
Aman	CSE
Neha	IT

R_2 (Dept, Subject)

Dept	Subject
CSE	DBMS
IT	Java
CSE	Java

Now join on Dept

Result:

Name	Dept	Subject
Aman	CSE	DBMS
Aman	CSE	Java
Neha	IT	Java

⇒ (Extra Tuple)

Summary :